

Task 5.1P

5.1P Activity Start (Activity 1-4)

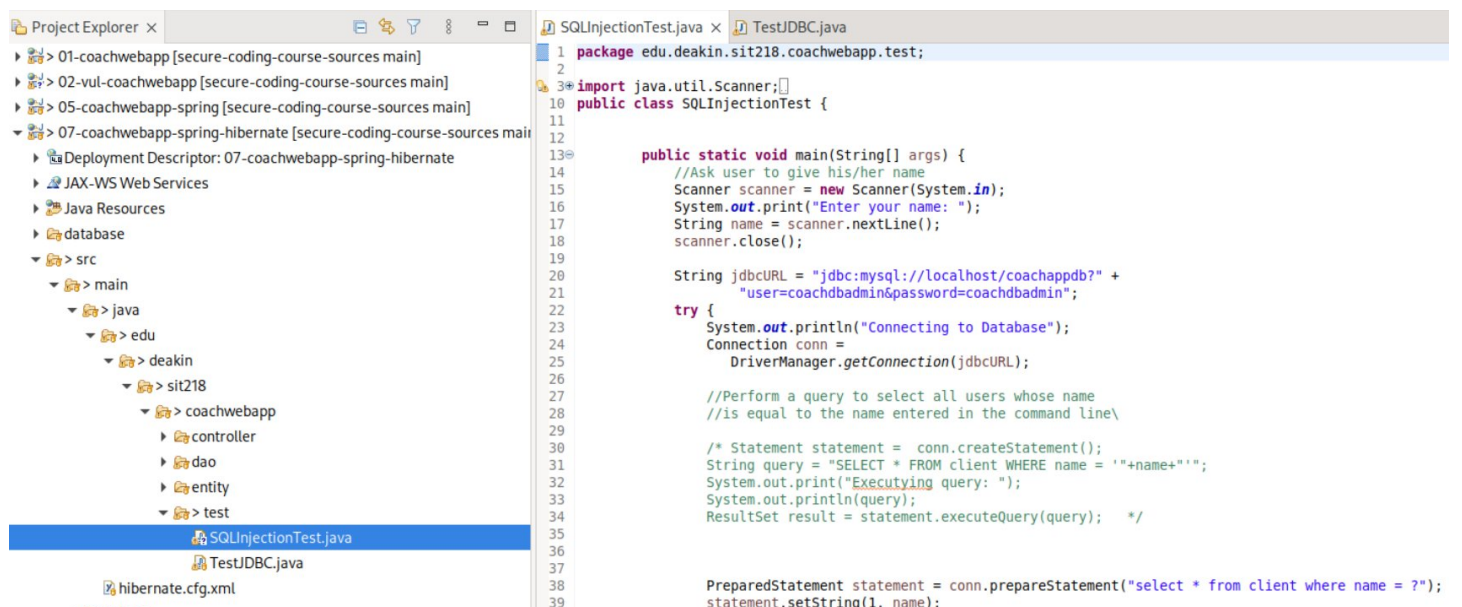
I started the mysql server.

```
(kali@kali)-[~]
$ sudo systemctl start mysql

(kali@kali)-[~]
$ sudo systemctl status mysql
● mariadb.service - MariaDB 11.8.6 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; disabled; preset: disabled)
   Active: active (running) since Thu 2026-08-27 10:30:04 EDT; 12s ago
     Invocation: 1075d392fc634ccd8cdd1d888eb1aff5
       Docs: man:mariadb(8)
            https://mariadb.com/kb/en/library/systemd/
   Process: 9283 ExecStartPre=/bin/sh -c [ ! -e /usr/bin/galera_recovery ] 66 VAR= || VAR="/usr/bin/galera_recovery"; [ $? -eq 0 ] 66 echo _WSREP_START_POSITION=$VAR > /run/mysqld/wsrep-start-
   Process: 9353 ExecStartPost=/bin/rm -f /run/mysqld/wsrep-start-position /run/mysqld/wsrep-new-cluster (code=exited, status=0/SUCCESS)
   Process: 9355 ExecStartPost=/etc/mysql/debian-start (code=exited, status=0/SUCCESS)
  Main PID: 9339 (mariabdd)
    Status: "Taking your SQL requests now..."
     Tasks: 14 (limit: 62166)
    Memory: 179M (peak: 183.9M)
       CPU: 1.120s
    CGroup: /system.slice/mariadb.service
            └─9339 /usr/sbin/mariabdd

Aug 27 10:30:03 kali mariabdd[9339]: 2026-08-27 10:30:03 0 [Note] Starting table crash recovery ...
Aug 27 10:30:03 kali mariabdd[9339]: 2026-08-27 10:30:03 0 [Note] Crash table recovery finished.
Aug 27 10:30:03 kali mariabdd[9339]: 2026-08-27 10:30:03 0 [Note] InnoDB: Buffer pool(s) load completed at 260827 10:30:03
Aug 27 10:30:03 kali mariabdd[9339]: 2026-08-27 10:30:03 0 [Note] Server socket created on IP: '127.0.0.1', port: '3306'.
Aug 27 10:30:04 kali mariabdd[9339]: 2026-08-27 10:30:04 0 [Note] mariabdd: Event Scheduler: Loaded 0 events
Aug 27 10:30:04 kali mariabdd[9339]: 2026-08-27 10:30:04 0 [Note] /usr/sbin/mariabdd: ready for connections.
Aug 27 10:30:04 kali mariabdd[9339]: Version: '11.8.6-MariaDB-0 from Debian' socket: '/run/mysqld/mysqld.sock' port: 3306 -- Please help get to 10k stars at https://github.com/MariaDB/Server
Aug 27 10:30:04 kali systemd[1]: Started mariadb.service - MariaDB 11.8.6 database server.
Aug 27 10:30:04 kali /etc/mysql/debian-start[9370]: Checking for insecure root accounts.
Aug 27 10:30:04 kali /etc/mysql/debian-start[9374]: Triggering mysam-recover for all MyISAM tables and aria-recover for all Aria tables
lines 1-27/27 (END)
```

I located the two said files as instructed.



```
package edu.deakin.sit218.coachwebapp.test;

import java.util.Scanner;

public class SQLInjectionTest {

    public static void main(String[] args) {
        //Ask user to give his/her name
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = scanner.nextLine();
        scanner.close();

        String jdbcURL = "jdbc:mysql://localhost/coachappdb?" +
            "user=coachdbadmin&password=coachdbadmin";

        try {
            System.out.println("Connecting to Database");
            Connection conn =
                DriverManager.getConnection(jdbcURL);

            //Perform a query to select all users whose name
            //is equal to the name entered in the command line\

            /* Statement statement = conn.createStatement();
            String query = "SELECT * FROM client WHERE name = '"+name+"'";
            System.out.print("Executing query: ");
            System.out.println(query);
            ResultSet result = statement.executeQuery(query); */

            PreparedStatement statement = conn.prepareStatement("select * from client where name = ?");
            statement.setString(1, name);
```

Uncommented the first block and commented the second block out as instructed.

```
30 Statement statement = conn.createStatement();
31 String query = "SELECT * FROM client WHERE name = '"+name+"'";
32 System.out.print("Executing query: ");
33 System.out.println(query);
34 ResultSet result = statement.executeQuery(query);
35
36
37
38 /* PreparedStatement statement = conn.prepareStatement("select * from client where name = ?");
39 statement.setString(1, name);
40 System.out.print(statement.toString() + "\n");
41 ResultSet result = statement.executeQuery(); */
```

Testing the value "Bob" shows that this works.

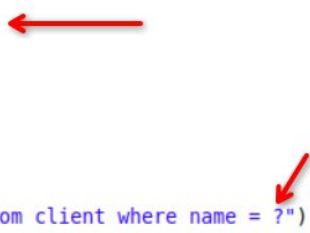
```
Enter your name: Bob
Connecting to Database
Executing query: SELECT * FROM client WHERE name = 'Bob'
2, Bob, 45, 1
Connection successful!
```

Because the input is badly handled and shoved directly into the SQL query, the injection attack worked.

```
Enter your name: Bob' OR 1=1 --'
Connecting to Database
Executing query: SELECT * FROM client WHERE name = 'Bob' OR 1=1 --'
1, Test, 34, 0
2, Bob, 45, 1
3, Tom, 24, 0
4, Paul, 56, 1
5, Test, 44, 1
6, Rolando, 20, 0
7, Rolando, 20, 0
8, John, 44, 3
Connection successful!
```

I commented out the unsafe query execution and uncommented the parameterized approach.

```
29
30
31      /* Statement statement = conn.createStatement();
32      String query = "SELECT * FROM client WHERE name = '"+name+"'";
33      System.out.print("Executing query: ");
34      System.out.println(query);
35      ResultSet result = statement.executeQuery(query); */
36
37
38      PreparedStatement statement = conn.prepareStatement("select * from client where name = ?");
39      statement.setString(1, name);
40      System.out.print(statement.toString() + "\n");
41      ResultSet result = statement.executeQuery();
```



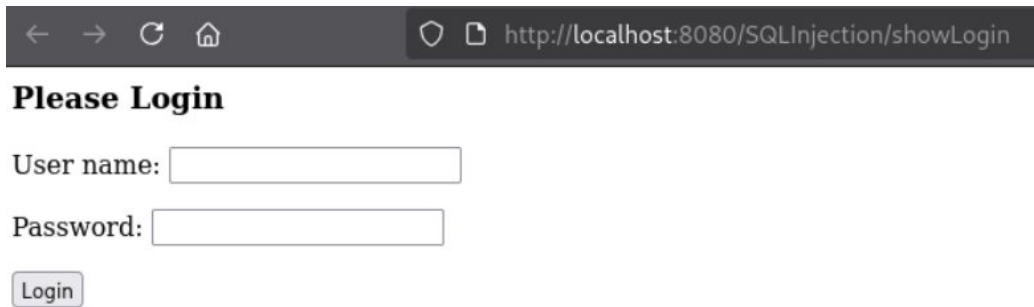
Now when you attempt the escape out the statement by performing some basic injection, it won't work as the query is prepared properly and the parameters are set to this statement as shown with the arrows in the above diagram.

```
Enter your name: Bob' OR 1=1 --'
Connecting to Database
com.mysql.cj.jdbc.ClientPreparedStatement: select * from client where name = 'Bob' OR 1=1 --'
Connection successful!
```

I updated the ClientDAOImpl.java as instructed.

```
60      session.beginTransaction();
61      // String result = username.substring(0,username.indexOf("'"));
62
63      //create the query
64      String hql = "from Client where username = '" + username + "'";
65      Query query = session.createQuery(hql);
```

I then started the web server.



← → ↻ 🏠 http://localhost:8080/SQLInjection/showLogin

Please Login

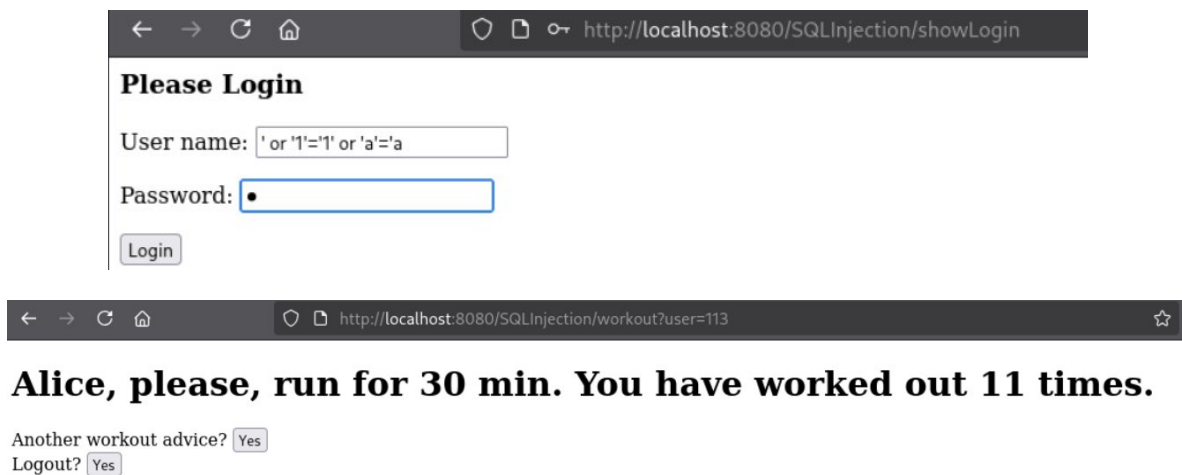
User name:

Password:

Login

5.1P Task Sheet – Part 1

After attempting some payloads, I was able to bypass the authentication with this username. The password can just be anything here



← → ↻ 🏠 http://localhost:8080/SQLInjection/showLogin

Please Login

User name:

Password:

Login

← → ↻ 🏠 http://localhost:8080/SQLInjection/workout?user=113

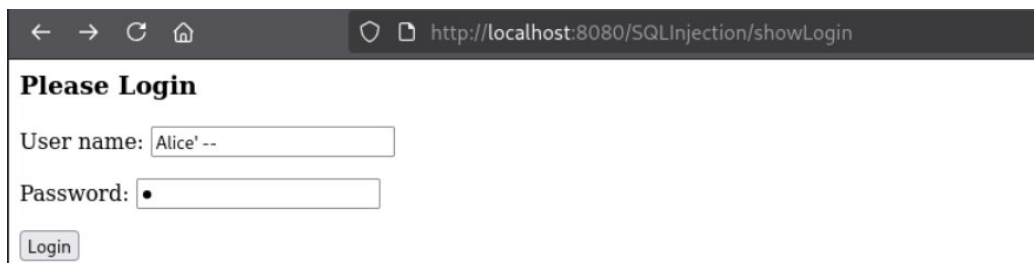
Alice, please, run for 30 min. You have worked out 11 times.

Another workout advice?

Logout?

Note that I also attempted ' or '1'='1' but that didn't work. Neither did commenting out worked. But this payload did: ' or '1'='1' or 'a'='a'.

Also, attempting to comment out like Alice' -- threw an exception and the web server returned a 404.



← → ↻ 🏠 http://localhost:8080/SQLInjection/showLogin

Please Login

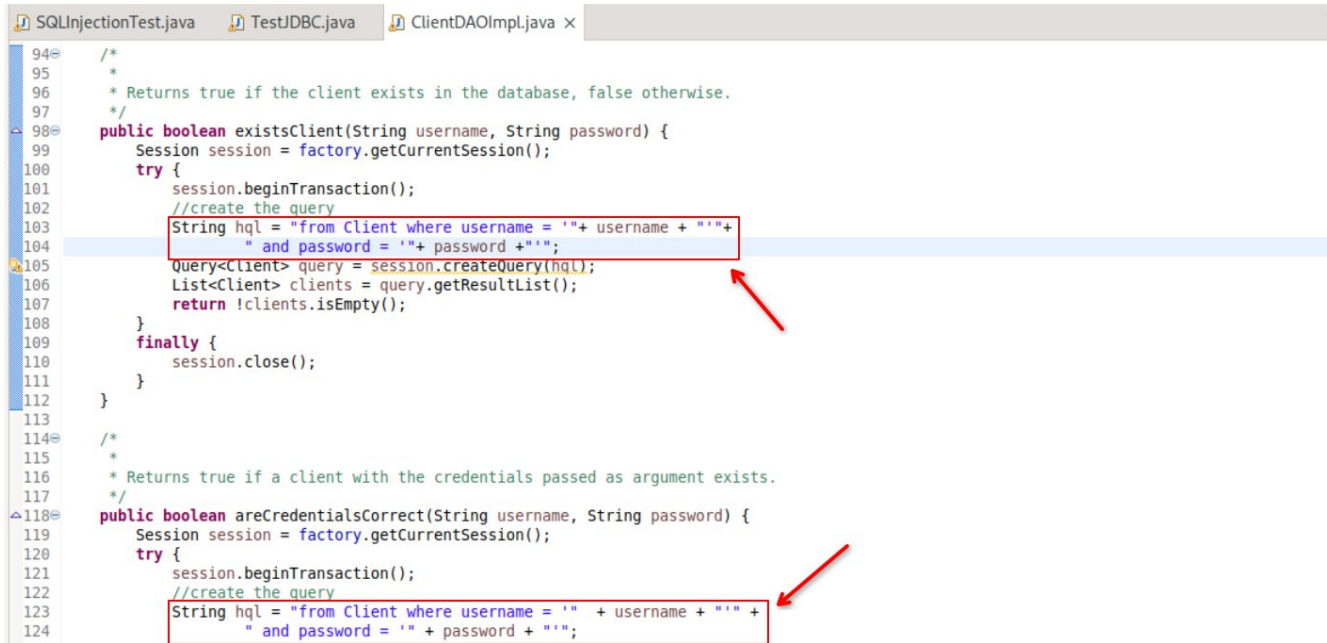
User name:

Password:

Login

```
Aug 27, 2026 11:38:30 AM org.apache.catalina.core.StandardWrapperValve invoke
SEVERE: Servlet.service() for servlet [dispatcher] in context with path [/SQLInjection] threw exception [Request processing failed; nested exception is java.lang.IllegalArgumentException: org.hib
org.hibernate.QueryException: expecting '', found '<EOF>' [from edu.deakin.sit218.coachwebapp.entity.Client where username = 'Alice' --' and password = 'a']]
    at org.hibernate.hql.internal.ast.QueryTranslatorImpl.doCompile(QueryTranslatorImpl.java:236)
    at org.hibernate.hql.internal.ast.QueryTranslatorImpl.compile(QueryTranslatorImpl.java:144)
    at org.hibernate.engine.query.spi.HQLQueryPlan.<init>(HQLQueryPlan.java:113)
    at org.hibernate.engine.query.spi.HQLQueryPlan.<init>(HQLQueryPlan.java:73)
    at org.hibernate.engine.query.spi.QueryPlanCache.getHQLQueryPlan(QueryPlanCache.java:162)
    at org.hibernate.internal.AbstractSharedSessionContract.getQueryPlan(AbstractSharedSessionContract.java:622)
    at org.hibernate.internal.AbstractSharedSessionContract.createQuery(AbstractSharedSessionContract.java:734)
    at org.hibernate.internal.AbstractSharedSessionContract.createQuery(AbstractSharedSessionContract.java:114)
    at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
    at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:77)
```

The highlighted lines of code here shows the vulnerable code.



```
94  /*
95  *
96  * Returns true if the client exists in the database, false otherwise.
97  */
98  public boolean existsClient(String username, String password) {
99      Session session = factory.getCurrentSession();
100      try {
101          session.beginTransaction();
102          //create the query
103          String hql = "from Client where username = '" + username + "'"
104                      + " and password = '" + password + "'";
105          Query<Client> query = session.createQuery(hql);
106          List<Client> clients = query.getResultList();
107          return !clients.isEmpty();
108      }
109      finally {
110          session.close();
111      }
112  }
113
114  /*
115  *
116  * Returns true if a client with the credentials passed as argument exists.
117  */
118  public boolean areCredentialsCorrect(String username, String password) {
119      Session session = factory.getCurrentSession();
120      try {
121          session.beginTransaction();
122          //create the query
123          String hql = "from Client where username = '" + username + "'"
124                      + " and password = '" + password + "'";
```

Now that I can actually see the query being executed, it makes sense as to why some payloads worked and why some failed.

' or '1'=1 failed because AND has higher priority than OR in HQL. The query was interpreted as:

username="" OR ('1='1' AND password='x')

The password didn't match and therefore, the AND section was false. `username=""` was also false because there was no user with an empty username, making the full condition false. Adding `OR 'a'='a'` fixed it because it added an unconditional OR true at the end. This made the full WHERE condition true no matter what the password is. The `--` comment didn't work because the application uses HQL rather than raw SQL. The HQL parser didn't recognise it as a comment, so it returned a parse error instead of ignoring the rest of the query.

This code is vulnerable because it directly concatenates the raw user input into the HQL query. This allows the user input to change the query logic instead of being treated only as a value. To fix this, we can use bind parameters, like: `query.setParameter("username", username)`. This will keep the input separate from the query structure by injecting it safely.

5.1P Task Sheet – Part 2

Task 1

The original function built the HQL query by directly concatenating the input username and password to the query string. This allowed user input to change the query logic, thus making the code vulnerable to SQL injection attacks.

To fix this, I first referred to the main() method in SQLInjectionTest.java. That example used a JDBC PreparedStatement with ? placeholders and setString() to bind the values separately. This means that the username and password are treated only as data and cannot become a part of the SQL syntax^[1].

The first possible approach was to follow the test file exactly and open a new connection using DriverManager.getConnection(). This was required in SQLInjectionTest.java because it was a standalone program without an existing database session^[2].

```
4 import java.sql.Connection;
5 import java.sql.DriverManager;
6 import java.sql.PreparedStatement;
7 import java.sql.ResultSet;
8 import java.sql.SQLException;

104 public boolean existsClient(String username, String password) {
105     boolean exists = false;
106     String jdbcURL = "jdbc:mysql://localhost:3306/sessiondb?user=coachdbadmin&password=coachdbadmin";
107     try {
108         Connection conn = DriverManager.getConnection(jdbcURL);
109
110         PreparedStatement statement = conn.prepareStatement("select * from user where username = ? and password = ?");
111         statement.setString(1, username);
112         statement.setString(2, password);
113         ResultSet result = statement.executeQuery();
114
115         exists = result.next();
116     } catch (SQLException ex) {
117         System.out.println("SQLException: " + ex.getMessage());
118         System.out.println("SQLState: " + ex.getSQLState());
119         System.out.println("VendorError: " + ex.getErrorCode());
120     }
121     return exists;
122 }
```

However, ClientDAOImpl already uses Hibernate ORM and has access to the current Session through factory.getCurrentSession(). Opening another connection inside existsClient() like done above would create a separate JDBC connection outside the Hibernate's existing connection and transaction management.. It would also require the DB connection string / URL and credentials to be added again, even though they were already configured for Hibernate.

```
3 import java.sql.Connection;
4 import java.sql.PreparedStatement;
5 import java.sql.ResultSet;
6 import java.sql.SQLException;
7
8 import java.util.List;
9
10 import org.hibernate.Session;
11 import org.hibernate.SessionFactory;
12 import org.hibernate.cfg.Configuration;
13 import org.hibernate.query.Query;
14 import org.hibernate.jdbc.Work; ←
15
```

```

103 public boolean existsClient(final String username, final String password) {
104     Session session = factory.getCurrentSession();
105     final boolean[] exists = { false };
106     try {
107         session.beginTransaction();
108         session.doWork(new Work() {
109             public void execute(Connection connection) throws SQLException {
110                 String sql = "select * from user where username = ? and password = ?";
111                 PreparedStatement statement = connection.prepareStatement(sql);
112                 statement.setString(1, username);
113                 statement.setString(2, password);
114                 ResultSet result = statement.executeQuery();
115                 exists[0] = result.next();
116             }
117         });
118         session.getTransaction().commit();
119         return exists[0];
120     }
121     finally {
122         session.close();
123     }
124 }

```

Instead, I used `session.doWork()` with the Hibernate's `Work` interface. `doWork()` provides access to the JDBC Connection already managed by the current Hibernate session^{[3][4]}. This let me use a normal `PreparedStatement` and `setString()` without opening another connection^[6].

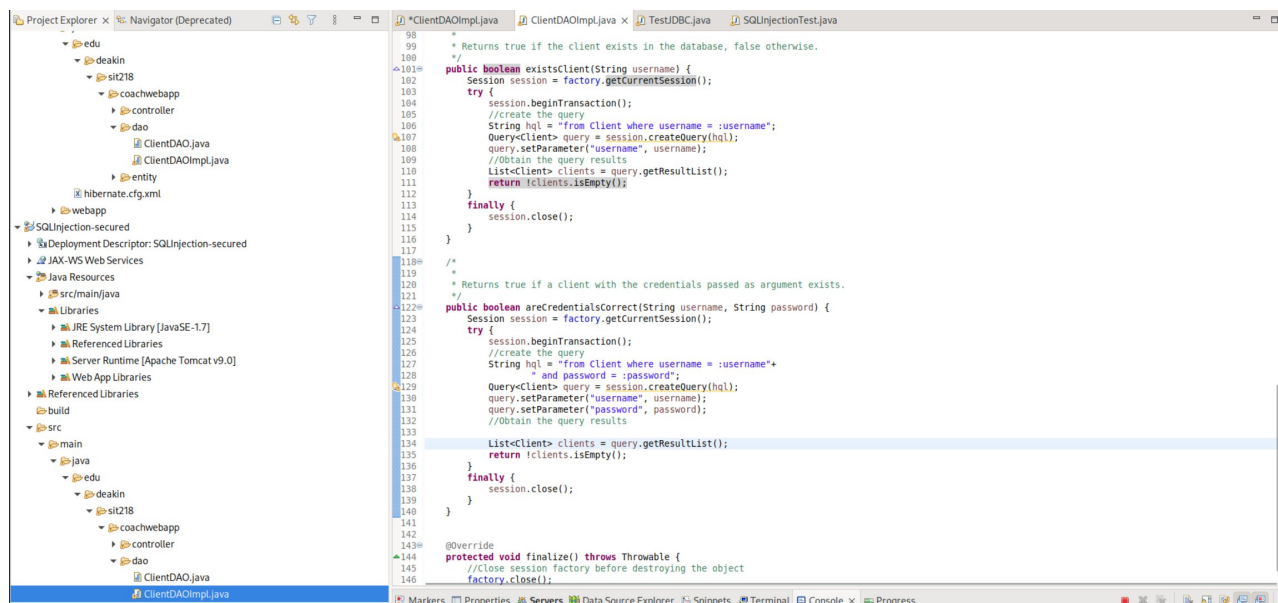
I also perform this operation within a Hibernate transaction using `session.beginTransaction()`. Once the query has completed successfully, I call `session.getTransaction().commit()` to complete the transaction. This keeps the JDBC query within the same session and transaction context already used by the DAO^[5].

The `Work.execute()` method does not return a value, so the result from `result.next()` is stored in the `exists` Boolean array and returned after `doWork()` finishes. Hibernate also provides `doReturningWork()` for cases where the JDBC operation needs to return a value directly^{[3][4]}.

This approach suits the application better because it works with Hibernate's existing session instead of creating a separate connection.

Task 2

I accessed the secured web app as instructed.



In the `existsClient()` and the `areCredentialsCorrect()` functions, the queries seem to be very secure. Instead of concatenating the username and the password to the HQL string directly – making it vulnerable, we use named parameters `:username` and `:password` as place holders. The actual values are then bound separately `query.setParameter()`. Hibernate converts this into a JDBC PreparedStatement with `?` placeholders and binds each value using the equivalent of `setString()`^[7]. This means the input is sent to the database only as data. Any injected quotes or operators remain part of the value and cannot change the query structure.

```
101 public boolean existsClient(String username) {
102     Session session = factory.getCurrentSession();
103     try {
104         session.beginTransaction();
105         //create the query
106         String hql = "from Client where username = :username";
107         Query<Client> query = session.createQuery(hql);
108         query.setParameter("username", username);
109         //Obtain the query results
110         List<Client> clients = query.getResultList();
111         return !clients.isEmpty();
112     }
113     finally {
114         session.close();
115     }
116 }
117
118 /*
119  *
120  * Returns true if a client with the credentials passed as argument exists.
121  */
122 public boolean areCredentialsCorrect(String username, String password) {
123     Session session = factory.getCurrentSession();
124     try {
125         session.beginTransaction();
126         //create the query
127         String hql = "from Client where username = :username"+
128             " and password = :password";
129         Query<Client> query = session.createQuery(hql);
130         query.setParameter("username", username);
131         query.setParameter("password", password);
132         //Obtain the query results
133
134         List<Client> clients = query.getResultList();
135         return !clients.isEmpty();
136     }
```

Below is the console output (further confirming the aforementioned point) when attempting to log in.

```
Tomcat v9.0 Server at localhost [Apache Tomcat] /opt/eclipse/eclipse/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.linux.x86_64_17.0.6.v20230204-1729/jre/bin/java (Aug 27, 2026, 8:09:00 PM) [pid: 18091]
Aug 28, 2026 12:35:40 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl configure
WARN: HHH10001002: Using Hibernate built-in connection pool (not for production use!)
Aug 28, 2026 12:35:40 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001005: using driver [com.mysql.cj.jdbc.Driver] at URL [jdbc:mysql://localhost:3306/sessiondb]
Aug 28, 2026 12:35:40 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001001: Connection properties: {password=****, user=coachdbadmin}
Aug 28, 2026 12:35:40 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001003: Autocommit mode: false
Aug 28, 2026 12:35:40 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PooledConnections <init>
INFO: HHH000115: Hibernate connection pool size: 1 (min=1)
Aug 28, 2026 12:35:40 AM org.hibernate.dialect.Dialect <init>
INFO: HHH000400: Using dialect: org.hibernate.dialect.MySQLDialect
Hibernate:
/*
from
  Client
where
  username = :username
  and password = :password */ select
  client0_.iduser as iduser1_0_,
  client0_.age as age2_0_,
  client0_.password as password3_0_,
  client0_.username as username4_0_,
  client0_.workouts as workouts5_0_
from
  user client0_
where
  client0_.username=?
  and client0_.password=?
Aug 28, 2026 12:35:41 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PoolState stop
INFO: HHH10001008: Cleaning up connection pool [jdbc:mysql://localhost:3306/sessiondb]
```


I tried a lot of common SQL Injection approaches, both manually trying to escape out of it as well as running other commonly used tools used in standard penetration testing methodologies.

```
(kali@kali)-[~]
$ wfuzz -t 1 -z file,/usr/share/wfuzz/wordlist/Injections/SQL.txt -d "username=Alice&password=FUZZ" -u http://localhost:8080/SQLInjection-secured/authenticateTheUser
/usr/lib/python3/dist-packages/wfuzz/__init__.py:34: UserWarning:Pycurl is not compiled against Openssl. Wfuzz might not work correctly when fuzzing SSL sites. Check Wfuzz's documentation for more information.
*****
* Wfuzz 3.1.0 - The Web Fuzzer
*****

Target: http://localhost:8080/SQLInjection-secured/authenticateTheUser
Total requests: 125

ID           Response      Lines      Word      Chars      Payload
-----
[1.10.6#stable]
https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 23:50:11 /2026-08-27/

[23:50:11] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('JSESSIONID=25A8049579F...F3E0009BB9'). Do you want to use those [Y/n] Y
[23:50:11] [INFO] testing if the target URL content is stable
[23:50:11] [INFO] target URL content is stable
[23:50:11] [WARNING] heuristic (basic) test shows that POST parameter 'username' might not be injectable
[23:50:11] [INFO] testing for SQL injection on POST parameter 'username'
[23:50:11] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[23:50:12] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (subquery - comment)'

[23:50:17] [INFO] checking if the injection point on POST parameter 'username' is a false positive
[23:50:17] [WARNING] false positive or unexploitable injection point detected
[23:50:17] [WARNING] POST parameter 'username' does not seem to be injectable
[23:50:17] [WARNING] heuristic (basic) test shows that POST parameter 'password' might not be injectable
[23:50:17] [INFO] testing for SQL injection on POST parameter 'password'
[23:50:17] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[23:50:18] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (subquery - comment)'
[23:50:19] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (comment)'
[23:50:19] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[23:50:19] [INFO] testing 'Boolean-based blind - Parameter replace (DUAL)'
[23:50:19] [INFO] testing 'Boolean-based blind - Parameter replace (DUAL - original value)'
[23:50:19] [INFO] testing 'Boolean-based blind - Parameter replace (CASE)'
[23:50:19] [INFO] testing 'Boolean-based blind - Parameter replace (CASE - original value)'
[23:50:19] [INFO] testing 'HAVING boolean-based blind - WHERE, GROUP BY clause'
[23:50:20] [INFO] testing 'Generic inline queries'
it is recommended to perform only basic UNION tests if there is not at least one other (potential) technique found. Do you want to reduce the number of requests? [Y/n] Y
[23:50:20] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[23:50:21] [INFO] testing 'Generic UNION query (random number) - 1 to 10 columns'
[23:50:21] [WARNING] POST parameter 'password' does not seem to be injectable
[23:50:21] [CRITICAL] all tested parameters do not appear to be injectable. Try to increase values for '--level'/'--risk' options if you wish to perform more tests. Also, you can try to rerun by providing a valid value for option '--string' as perhaps the string you have chosen does not match exclusively True responses. If you suspect that there is some kind of protection mechanism involved (e.g. WAF) maybe you could try to use option '--tamper' (e.g. '--tamper=space2comment') and/or switch '--random-agent'
[23:50:21] [WARNING] HTTP error codes detected during run:
404 (Not Found) - 489 times

[*] ending @ 23:50:21 /2026-08-27/
```

No injection approach worked across hundreds of payloads, a differential fuzz of an entire dedicated SQLi wordlist, and a full sqlmap sweep. Because of the secure coding practices followed here, the inject input is always treated as a literal value and never as more SQL syntax.

However, I found the credentials hardcoded in the hibernate.cfg.xml file.

```
9      <!-- JDBC Database connection settings -->
10     <property
11       name="connection.driver_class">com.mysql.cj.jdbc.Driver</property>
12     <property
13       name="connection.url">jdbc:mysql://localhost:3306/sessiondb</property>
14     <property name="connection.username">coachdbadmin</property>
15     <property name="connection.password">coachdbadmin</property>
16
17     <!-- JDBC connection pool settings using built-in test pool -->
```

Using these credentials and by inspecting the code, I was able to extract the record for user Alice.

```
mysql -h localhost -u coachdbadmin -pcoachdbadmin
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3751
Server version: 11.8.6-MariaDB-6 from Debian -- Please help get to 10k stars at https://github.com/MariaDB/Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> SELECT * FROM sessiondb.user WHERE username='Alice';
+-----+-----+-----+-----+-----+
| iduser | username | password | age | workouts |
+-----+-----+-----+-----+-----+
| 113 | Alice | alice | 66 | 13 |
+-----+-----+-----+-----+-----+
1 row in set (0.000 sec)

MariaDB [(none)]>
```

The password was not hashed (+salted) and has just been stored as plain text. Using “alice” as the password, I was able to log in.

← → ↺ ↻ http://localhost:8080/SQLInjection-secured/workout?user=113 130% ☆

Alice, please, run for 30 min. You have worked out 13 times.

Another workout advice?

Logout?

Tomcatv9.0 Server at localhost [Apache Tomcat] /opt/eclipse/eclipse/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.linux.x86_64.17.0.6.v20230204-1729/jre/bin/java (Aug 27, 2026, 8:09:00 PM) [pid: 18091]

```
Aug 28, 2026 12:45:12 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl configure
WARN: HHH10001002: Using Hibernate built-in connection pool (not for production use!)
Aug 28, 2026 12:45:12 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001005: using driver [com.mysql.cj.jdbc.Driver] at URL [jdbc:mysql://localhost:3306/sessiondb]
Aug 28, 2026 12:45:12 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001001: Connection properties: {password=****, user=coachdbadmin}
Aug 28, 2026 12:45:12 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001003: Autocommit mode: false
Aug 28, 2026 12:45:12 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PooledConnections <init>
INFO: HHH000015: Hibernate connection pool size: 1 (min=1)
Aug 28, 2026 12:45:12 AM org.hibernate.dialect.Dialect <init>
INFO: HHH000400: Using dialect: org.hibernate.dialect.MySQLDialect
Hibernate:
/*
from
  Client
where
  username = :username
  and password = :password */ select
  client0_.iduser as iduser1_0_,
  client0_.age as age2_0_,
  client0_.password as password3_0_,
  client0_.username as username4_0_,
  client0_.workouts as workouts5_0_
from
  user client0_
where
  client0_.username=?
  and client0_.password=?
Hibernate:
/*
from
  Client
where
  username = :username */ select
  client0_.iduser as iduser1_0_,
  client0_.age as age2_0_,
  client0_.password as password3_0_,
  client0_.username as username4_0_,
  client0_.workouts as workouts5_0_
from
  user client0_
where
  client0_.username=?
Aug 28, 2026 12:45:12 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl configure
WARN: HHH10001002: Using Hibernate built-in connection pool (not for production use!)
Aug 28, 2026 12:45:12 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001005: using driver [com.mysql.cj.jdbc.Driver] at URL [jdbc:mysql://localhost:3306/sessiondb]
```

Tomcat v9.0 Server at localhost [Apache Tomcat] /opt/eclipse/eclipse/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.linux.x86_64_17.0.6.v20230204-1729/jre/bin/java (Aug 27, 2026, 8:09:00 PM) [pid: 18091]

```
INFO: HHH10001005: using driver [com.mysql.cj.jdbc.Driver] at URL [jdbc:mysql://localhost:3306/sessiondb]
Aug 28, 2026 12:45:12 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001001: Connection properties: {password=****, user=coachdbadmin}
Aug 28, 2026 12:45:12 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001003: Autocommit mode: false
Aug 28, 2026 12:45:12 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PooledConnections <init>
INFO: HHH000115: Hibernate connection pool size: 1 (min=1)
Aug 28, 2026 12:45:12 AM org.hibernate.dialect.Dialect <init>
INFO: HHH000400: Using dialect: org.hibernate.dialect.MySQLDialect
Hibernate:
  select
    client0_.iduser as iduser1_0_0_,
    client0_.age as age2_0_0_,
    client0_.password as password3_0_0_,
    client0_.username as username4_0_0_,
    client0_.workouts as workouts5_0_0_
  from
    user client0_
  where
    client0_.iduser=?
Aug 28, 2026 12:45:12 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl configure
WARN: HHH10001002: Using Hibernate built-in connection pool (not for production use!)
Aug 28, 2026 12:45:12 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001005: using driver [com.mysql.cj.jdbc.Driver] at URL [jdbc:mysql://localhost:3306/sessiondb]
Aug 28, 2026 12:45:12 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001001: Connection properties: {password=****, user=coachdbadmin}
Aug 28, 2026 12:45:12 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001003: Autocommit mode: false
Aug 28, 2026 12:45:12 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PooledConnections <init>
INFO: HHH000115: Hibernate connection pool size: 1 (min=1)
Aug 28, 2026 12:45:12 AM org.hibernate.dialect.Dialect <init>
INFO: HHH000400: Using dialect: org.hibernate.dialect.MySQLDialect
Hibernate:
/* update
  edu.deakin.sit218.coachwebapp.entity.Client */ update
  user
  set
    age=?,
    password=?,
    username=?,
    workouts=?
  where
    iduser=?
Aug 28, 2026 12:45:13 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PoolState stop
INFO: HHH10001008: Cleaning up connection pool [jdbc:mysql://localhost:3306/sessiondb]
Aug 28, 2026 12:45:13 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PoolState stop
INFO: HHH10001008: Cleaning up connection pool [jdbc:mysql://localhost:3306/sessiondb]
Aug 28, 2026 12:45:13 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PoolState stop
```

Execution flow^[8]:

When the web server receives a POST /authenticateTheUser request, web.xml sends it to Spring's DispatcherServlet. This is the front controller. It handles the incoming requests for the application.

When the application starts, the DispatcherServlet will read spring-configuration.xml. The component scan finds the controller classes. The view resolver also connects view names to JSP files inside /WEB-INF/view/.

The request will then go to LoginController. Spring reads the username and password using @RequestParam, then calls the authenticateTheUser() method.

The controller passes the details to ClientDAOImpl.areCredentialsCorrect(). This method uses named parameters and query.setParameter() instead of adding the input directly into the HQL query. Hibernate will use a parameterised query, so the input cannot change the query structure.

The database connection details comes from hibernate.cfg.xml. Hibernate runs the query against the user table and maps the result into a Client object.

If the details match, the controller redirects the user to the workout page. If it doesn't match, an error message is added to the model. The plain-login.jsp page reads this message and shows it to the user.

In summary:

Browser submits username and password --> web.xml sends the request to DispatcherServlet --> LoginController receives /authenticateTheUser --> ClientDAOImpl checks the credentials using HQL and setParameter() --> Hibernate runs a parameterised PreparedStatement using hibernate.cfg.xml --> MySQL checks the user table --> Hibernate maps the result to a Client object --> controller checks the result --> plain-login.jsp displays \${errorMessage} or redirects to /workout

Task 3

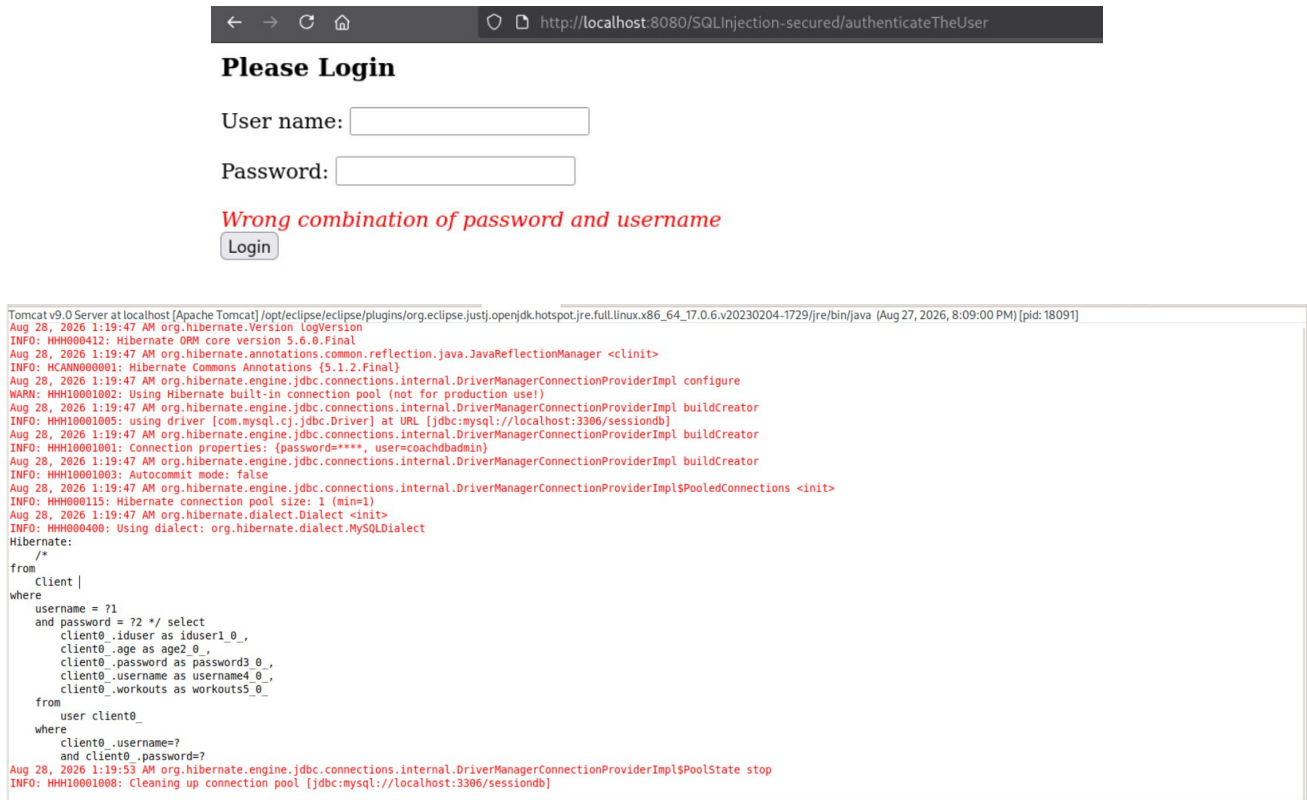
Task 2 used named parameters like :username and :password. The values were bound by name using `setParameter("username", ...)`.

This version uses JPA style ordinal parameters instead. ?1 is used for the username and ?2 is used for the password. The values are then bound by position using `setParameter(1, username)` and `setParameter(2, password)`. The security doesn't change. Hibernate still creates a JDBC PreparedStatement and passes the values separately. The input is treated as data, so it can't change the query structure.

The main difference is how the parameters are identified. Named parameters use a name, while ordinal parameters use a number like seen here. The numbers must match correctly. If ?1 and ?2 are mixed up, the wrong values could be checked. It still won't cause SQL injection as long as the values are bound instead of being added directly to the query.

```
197-  /*
198-   *
199-   * Returns true if the client exists in the database, false otherwise.
200-   */
201-  public boolean existsClient(String username) {
202-      Session session = factory.getCurrentSession();
203-      try {
204-          session.beginTransaction();
205-          //create the query using a JPA-style ordinal parameter (?1)
206-          //instead of a named parameter
207-          String hql = "from Client where username = ?1";
208-          Query<Client> query = session.createQuery(hql);
209-          query.setParameter(1, username);
210-          //Obtain the query results
211-          List<Client> clients = query.getResultList();
212-          return !clients.isEmpty();
213-      }
214-      finally {
215-          session.close();
216-      }
217-  }
218-
219-  /*
220-   *
221-   * Returns true if a client with the credentials passed as argument exists.
222-   */
223-  public boolean areCredentialsCorrect(String username, String password) {
224-      Session session = factory.getCurrentSession();
225-      try {
226-          session.beginTransaction();
227-          //create the query using JPA style ordinal parameters (?1, ?2)
228-          //instead of named parameters
229-          String hql = "from Client where username = ?1 and password = ?2";
230-          Query<Client> query = session.createQuery(hql);
231-          query.setParameter(1, username);
232-          query.setParameter(2, password);
233-          //Obtain the query results
234-
235-          List<Client> clients = query.getResultList();
236-          return !clients.isEmpty();
237-      }
238-      finally {
239-          session.close();
240-      }
241-  }
242-  }
```


Attempting a SQLi payload still fails as expected.

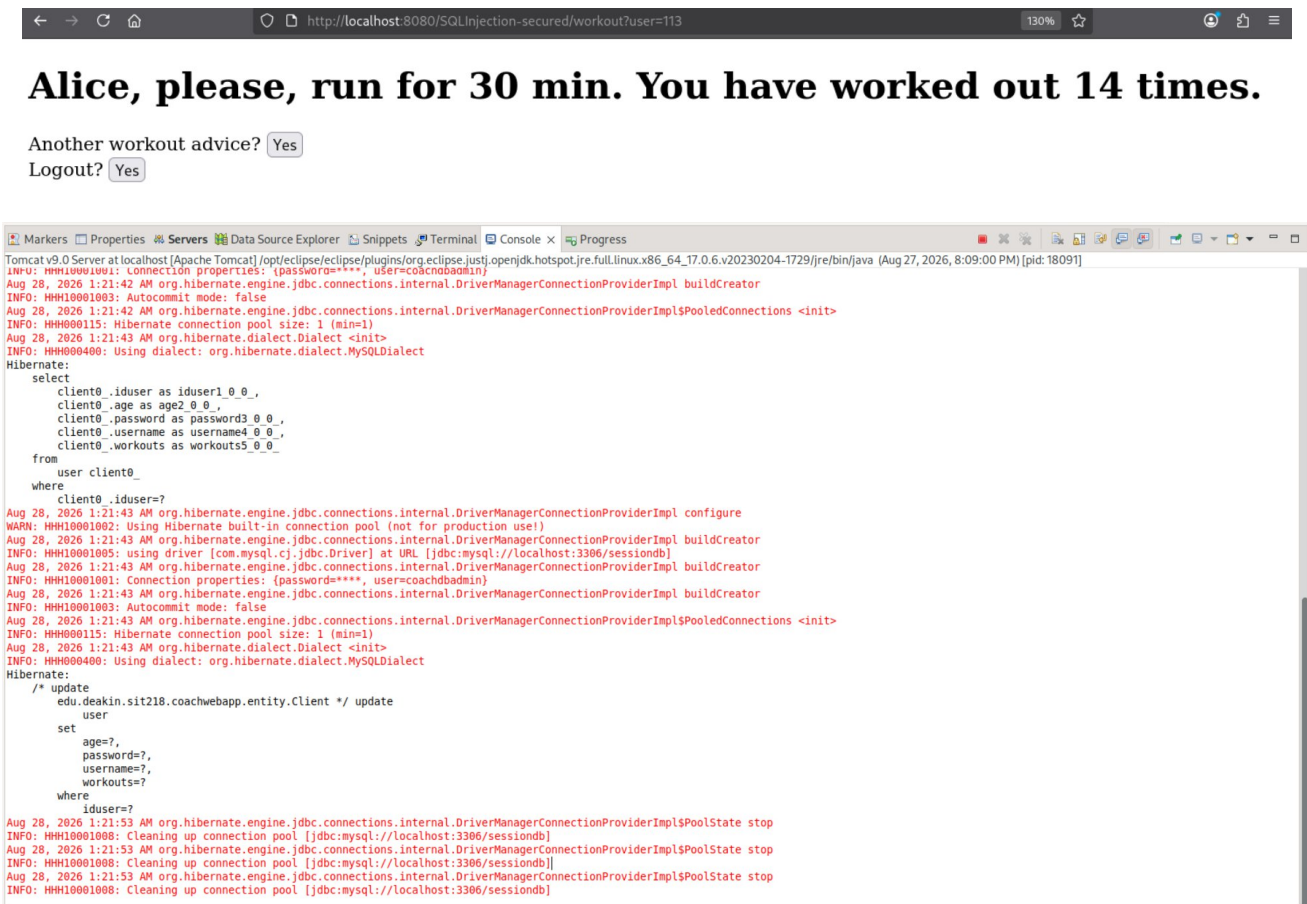


The screenshot shows a web browser at <http://localhost:8080/SQLInjection-secured/authenticateTheUser>. The page has a heading "Please Login" and two input fields for "User name:" and "Password:". Below the fields is a red error message: "Wrong combination of password and username". A "Login" button is at the bottom.

Below the browser window is a terminal window showing the server logs for Tomcat v9.0. The logs include the following SQL query:

```
from
Client |
where
  username = ?1
  and password = ?2 */ select
  client0_.iduser as iduser1_0_,
  client0_.age as age2_0_,
  client0_.password as password3_0_,
  client0_.username as username4_0_,
  client0_.workouts as workouts5_0_
from
  user client0_
where
  client0_.username=?
  and client0_.password=?
Aug 28, 2026 1:19:53 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PoolState stop
INFO: HHH10001008: Cleaning up connection pool [jdbc:mysql://localhost:3306/sessiondb]
```

However, you can still log in using the correct credentials.



The screenshot shows a web browser at <http://localhost:8080/SQLInjection-secured/workout?user=113>. The page displays the message: "Alice, please, run for 30 min. You have worked out 14 times." Below this are two buttons: "Another workout advice?" (with a "Yes" button) and "Logout?" (with a "Yes" button).

Below the browser window is a terminal window showing the server logs for Tomcat v9.0. The logs include the following SQL query:

```
select
  client0_.iduser as iduser1_0_,
  client0_.age as age2_0_,
  client0_.password as password3_0_,
  client0_.username as username4_0_,
  client0_.workouts as workouts5_0_
from
  user client0_
where
  client0_.iduser=?
Aug 28, 2026 1:21:43 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl configure
WARN: HHH10001002: Using Hibernate built-in connection pool (not for production use!)
Aug 28, 2026 1:21:43 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001005: using driver [com.mysql.cj.jdbc.Driver] at URL [jdbc:mysql://localhost:3306/sessiondb]
Aug 28, 2026 1:21:43 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001001: Connection properties: {password=****, user=coachdbadmin}
Aug 28, 2026 1:21:43 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001003: Autocommit mode: false
Aug 28, 2026 1:21:43 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PooledConnections <init>
INFO: HHH000115: Hibernate connection pool size: 1 (min=1)
Aug 28, 2026 1:21:43 AM org.hibernate.dialect.Dialect <init>
INFO: HHH000400: Using dialect: org.hibernate.dialect.MySQLDialect
Hibernate:
/*
update
  edu.deakin.sit218.coachwebapp.entity.Client */ update
  user
set
  age=?,
  password=?,
  username=?,
  workouts=?
where
  iduser=?
Aug 28, 2026 1:21:53 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PoolState stop
INFO: HHH10001008: Cleaning up connection pool [jdbc:mysql://localhost:3306/sessiondb]
Aug 28, 2026 1:21:53 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PoolState stop
INFO: HHH10001008: Cleaning up connection pool [jdbc:mysql://localhost:3306/sessiondb]
Aug 28, 2026 1:21:53 AM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PoolState stop
INFO: HHH10001008: Cleaning up connection pool [jdbc:mysql://localhost:3306/sessiondb]
```

References

- [1] Oracle, "Using prepared statements," The Java Tutorials. [Online]. Available: <https://docs.oracle.com/javase/tutorial/jdbc/basics/prepared.html>. [Accessed: Aug. 28, 2026].
- [2] Oracle, "Establishing a connection," The Java Tutorials. [Online]. Available: <https://docs.oracle.com/javase/tutorial/jdbc/basics/connecting.html>. [Accessed: Aug. 28, 2026].
- [3] Hibernate ORM, "Work interface," Hibernate ORM 5.6 JavaDocs. [Online]. Available: <https://docs.hibernate.org/orm/5.6/javadocs/org/hibernate/jdbc/Work.html>. [Accessed: Aug. 28, 2026].
- [4] Hibernate ORM, "SharedSessionContract interface," Hibernate ORM 5.6 JavaDocs. [Online]. Available: <https://docs.hibernate.org/orm/5.6/javadocs/org/hibernate/SharedSessionContract.html>. [Accessed: Aug. 28, 2026].
- [5] Hibernate ORM, "Transaction interface," Hibernate ORM 5.6 JavaDocs. [Online]. Available: <https://docs.hibernate.org/orm/5.6/javadocs/org/hibernate/Transaction.html>. [Accessed: Aug. 28, 2026].
- [6] KeatsPeeks, "Answer to 'session.connection() deprecated on Hibernate?'," Stack Overflow, Aug. 19, 2010, updated Aug. 15, 2022. [Online]. Available: <https://stackoverflow.com/questions/3526556/session-connection-deprecated-on-hibernate>. [Accessed: Aug. 28, 2026].
- [7] Hibernate ORM, "Hibernate Query API," Hibernate ORM 5.6.15.Final User Guide, sec. 15.2.2. [Online]. Available: https://docs.hibernate.org/orm/5.6/userguide/html_single/. [Accessed: Aug. 28, 2026].
- [8] Spring Framework, "Web MVC framework," Spring Framework Reference Documentation, ver. 4.1.7.RELEASE. [Online]. Available: <https://docs.spring.io/spring-framework/docs/4.1.7.RELEASE/spring-framework-reference/html/mvc.html>. [Accessed: Aug. 28, 2026].
- [9] K. Sierra and B. Bates, SCJP Sun Certified Programmer for Java 6 Study Guide: Exam 310-065. New York, NY, USA: McGraw-Hill Professional, 2008.